

UNIT-3

1.Iterations and Loops

1.1 While Loop

A while loop in C++ is a control statement that allows code to be executed repeatedly based on a condition. It is an entry-controlled loop because the condition is checked before the loop body executes.

Syntax

```
while (condition) {  
    // statements  
}
```

Example – Print numbers 1 to 5

```
#include <iostream>  
using namespace std;  
int main(){  
    int i = 1;  
    while (i <= 5){  
        cout << i << endl;  
        i++;  
    }  
    return 0;  
}
```

Output:

```
1  
2  
3  
4  
5
```

1.2 Do-While Loop

A do-while loop in C++ is a control statement that executes the loop body at least once and then checks the condition. It is an exit-controlled loop.

Syntax

```
do {  
    // statements  
} while (condition);
```

Example – Print numbers 1 to 5

```
#include <iostream>
using namespace std;
int main(){
    int i = 1;
    do {
        cout << i << endl;
        i++;
    } while (i <= 5);
    return 0;
}
```

Output:

```
1
2
3
4
5
```

1.3 For Loop

A for loop in C++ is used when the number of iterations is known in advance. It combines initialization, condition, and increment/decrement in a single line.

Syntax

```
    for (initialization; condition; increment) {
        // statements
    }
```

Example – Print first 5 natural numbers

```
#include <iostream>
using namespace std;
int main(){
    for (int i = 1; i <= 5;
        i++){ cout << i << endl;
    }
    return 0;
}
```

Output:

```
1
2
3
4
```

1.3.1 Multiple Loop Variables

In for loops in C++, multiple variables can be controlled simultaneously using commas.

Example – Print pairs of numbers

```
#include <iostream>
using namespace std;
int main(){
    for (int i = 1, j = 5; i <= 5; i++, j--){
        cout << "i = " << i << ", j = " << j << endl;
    }
    return 0;
}
```

Output:

```
i = 1, j = 5
i = 2, j = 4
i = 3, j = 3
i = 4, j = 2
i = 5, j = 1
```

1.3.2 Break and Continue Statements

break is used to terminate the loop immediately.

continue skips the current iteration and moves to the next one.

Example – Break and Continue

```
#include <iostream>
using namespace std;
int main(){
    for (int i = 1; i <= 5;
        i++){ if (i == 3){
        continue; // skips 3
        }
        if (i == 5){
        break; // stops at 5
        }
        cout << i << endl;
    }
    return 0;}
```

Output: 1
2
4

1.4 Nested Iteration

A nested loop in C++ means using one loop inside another. Commonly used for matrices, patterns, etc.

Example – Print a 3x3 star pattern

```
#include <iostream>
using namespace std;
int main(){
    for (int i = 1; i <= 3; i++){
        for (int j = 1; j <= 3;
            j++){ cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

Output:

```
* * *
* * *
* * *
```

2. Taking input on coding platforms-2

2.1 Single value in a single line

Description: Only one value is given as input in a line.

Example: Input → 10

Code:

```
int x;

cin >> x; // Reads a single integer
```

2.2 Multiple known values in a single line:

Description: Several values are given in one line, and we already know how many there are.

Example: Input → 10 20 30

Code:

```
int a, b, c;  
  
cin >> a >> b >> c; // Reads 3 integers from one line
```

2.3 Others (variations)

2.3.1 Values in multiple lines:

INPUT: 5
7

Code:

```
int x, y;  
  
cin >> x >> y; // Still works, cin ignores line breaks
```

2.3.2 Different data types

INPUT:

23 A 12.5

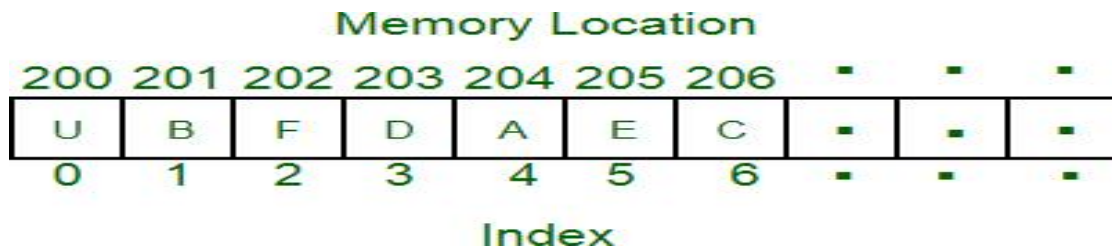
CODE:

```
int age; char grade; float marks; cin >> age >>  
grade >> marks;
```

3. ARRAYS

3.1 Introduction to Arrays

An array is a collection of items of same data type stored at contiguous memory locations.



Significant points about Array:

- I. It is a group of variables of similar data types referred to by a single element.
- II. Its elements are stored in a contiguous memory location.
- III. Array elements are always counted from zero (0) onward.
- IV. Array elements can be accessed using the position of the element in the array.
- V. The array can have one or more dimensions.

Syntax:

```
int arr[ 5 ] = { 1 , 2 , 3 , 4 , 5 };
```

3.2 Types of Arrays :

Single dimensional arrays

Multidimensional arrays.

3.2.1 Single Dimensional Arrays:

Single dimensional array or 1-D array is the simplest form of arrays that can be found. This type of array consists of elements of similar types and these elements can be accessed through their indices.

E.g. One-dimensional arrays `int mark[5] = {19, 10, 8, 17, 9};`

Arrays have 0 as the first index not 1. In this example, `mark[0]` If the size of an array is `n`, to access the last element, `(n-1)` index is used.

In this example, `mark[4]` is 9.

Sample Program Of Single Dimensional Array

```
#include <iostream>

int main() {

    // Declare and initialize an integer array

    int numbers[5] = {10, 20, 30, 40, 50};

    // Access and print elements using their index

    std::cout << "Element at index 0: " << numbers[0] << std::endl;

    std::cout << "Element at index 2: " << numbers[2] << std::endl;

    // Change an element's value numbers[1] = 25;

    std::cout << "Modified element at index 1: " << numbers[1] <<
    std::endl;

    return 0;
}
```

3.2.2 Multidimensional arrays:

you can create an array of arrays known as multidimensional array.

For example, `float x[3][4];` Here, `x` is a two-dimensional (2d) array. The array can hold 12 elements. You can think the array as table with 3 row and each row has 4 column.

Similarly, you can declare a three-dimensional (3d) array. For example, `float y[2][4][3];` Here, The array `y` can hold 24 elements.

columns				rows
1	22	33	44	
2	12	34	56	
3	10	5	20	
1 2 3 4				
Two-dimensional Array with 3 rows and 4 columns				

Two dimensional array (2D array) is organized as matrices which can be represented as the collection of rows and columns. The syntax to declare the 2D array is given below:

Syntax: data_type array_name[rows][columns];

Sample Program Of Multi-Dimensional Array

```
#include <iostream>

using namespace std;

int main() {

    // Declare and initialize a 2x3 integer array
    int matrix[2][3] = {{1, 2, 3}, {4, 5, 6}};

    // Access and print elements
    cout << "Element at matrix[0][0]: " << matrix[0][0] << endl;
    cout << "Element at matrix[1][2]: " << matrix[1][2] << endl;

    // Iterate through the 2D array
    cout << "Matrix elements:" << endl;

    for (int i = 0; i < 2; ++i) {

        // Rows
        for (int j = 0; j < 3; ++j)
        { // Columns
            cout << matrix[i][j] << " ";

        }

        cout << endl;

    }

    return 0;

}
```


Some of the most common Applications of arrays are as follows:

- i. Contacts on a mobile device.
- ii. Storing information in a table format.
- iii. Matrix storage and binary tree elements with fixed counts.
- iv. Image processing employs 2D arrays, also referred to as matrices.
- v. Additionally, it is utilized in speech processing, where every speech signal is an array.
- vi. Another multidimensional array of pixels is your viewing screen.
- vii. The names of books in a library management system.
- viii. Booking tickets online.
- ix. To keep track of a chessboard's potential moves.
- x. To keep pictures of a particular size.

4 Introduction to Dynamically Growable Arrays(Vectors)

In C++, arrays are fixed in size – once declared, their size cannot change at runtime. But in many real-world applications, the amount of data is not known in advance. For such cases, dynamically growable arrays are required.

4.1 What are Vectors?

A **vector** in C++ is a part of the **Standard Template Library (STL)**.

It is essentially a **dynamic array** that can grow or shrink in size automatically at runtime.

Vectors are defined in the header file **<vector>**.

Characteristics of Vectors:

- I. **Dynamic sizing** – size increases or decreases as elements are added or removed.
- II. **Contiguous memory allocation** – like arrays, elements are stored in adjacent memory.
- III. **Random access** – supports indexing (`v[i]`) just like arrays.
- IV. **Efficient insertion/deletion at the end** – `push_back()` and `pop_back()` are very fast.
- V. **Automatic memory management** – resizing is handled internally.

4.2 Commonly Used Functions:

- i. `v.push_back(x)` → Adds element at the end.
- ii. `v.pop_back()` → Removes last element.
- iii. `v.size()` → Returns number of elements.
- iv. `v.capacity()` → Returns allocated memory size.
- v. `v.front()` / `v.back()` → Access first/last element.
- vi. `v.clear()` → Removes all elements.

5. Taking inputs on coding platforms - 3:

5.1 Multiple unknown values in a single line:

Description: Number of values is not fixed. Example:

Input : 2 4 6 8 10

Code:

```
#include <iostream>
#include <sstream>
#include <string>
using namespace std;
int main()
{
    string line;
    getline(cin, line); // read whole line
    stringstream ss(line);
    int x;
    while (ss >> x) {
        // keep extracting integers
        cout << x << " ";
    }
    return 0; }
```

5.2 Multiple unknown values in multiple lines

Description: Number of values is unknown, and spread across lines. Often used when input ends at EOF (end of file).

INPUT: 5
10
15
20

CODE:

```
#include <iostream>

using namespace std;

int main(){

    int x;

    while (cin >> x)

    { // keeps running until EOF

        cout << x << " ";

    }

    return 0;}
```

Key Takeaways:

- i. cin ignores spaces and line breaks → makes input flexible.
- ii. Use getline() + stringstream for unknown numbers in a single line.
- iii. Use a while loop with cin for unknown numbers across multiple lines (EOF input).
- iv. Coding platforms often use space-separated or newline-separated inputs, so both cin and getline methods are important.

